

1 Abstract

This study presents a comprehensive quantitative analysis of High-Performance Liquid Chromatography (HPLC) data to assess system stability, gradient alignment, and product purity. The workflow processed raw UV signals (280 nm and 260 nm) from 11 chromatographic runs, employing Asymmetric Least Squares (AsLS) baseline correction to mitigate negative signal drift and excluding non-physical pre-run intervals. Key findings indicate successful system stability with distinct pressure anomaly detection, though a significant stratification in purity profiles was observed based on column chemistry. While Cation Exchange (CM, SP) columns demonstrated low 260/280 ratios indicative of high protein purity, Anion Exchange (DEAE, Q) columns exhibited elevated ratios, flagging 479 high-risk fractions for potential nucleic acid contamination. These results underscore the necessity of column-specific purity monitoring and validate the robustness of the integrated area-based quantification method.

2 Introduction

The accurate quantification of biomolecules via HPLC relies heavily on the integrity of UV absorbance signals and the stability of the chromatographic system. In this analysis, the primary objective was to derive robust abundance and purity metrics from high-frequency signal data, specifically focusing on the 280 nm wavelength for protein abundance and the 260 nm wavelength for nucleic acid contamination assessment. A critical challenge in such datasets is the presence of baseline drift and non-physical data points, such as negative volume intervals representing pre-run phases. Furthermore, the dataset exhibited a significant metadata gap, with approximately 75% of rows lacking specific sample codes, necessitating an aggregation strategy based on run numbers and column types rather than individual samples. This report details the methodology used to preprocess these signals, integrate peak areas, and evaluate system health, ultimately aiming to identify systematic purity issues across different chromatographic modes.

3 Methodology

The data analysis workflow consisted of three primary steps. First, signal preprocessing involved filtering the dataset to exclude all rows with negative 'volume_ml' values, effectively removing pre-run intervals. Asymmetric Least Squares (AsLS) baseline correction was applied to both 'UV_1_280_mAU' and 'UV_2_260_mAU' signals using parameters $\lambda \approx 10^6 - 10^7$ and $p \approx 10^{-4} - 10^{-5}$ to correct for a documented negative drift of approximately 7.95 mAU. Rows with missing 'Fraction_unique_ID' were retained for system stability monitoring but excluded from fraction-specific integration. Second, peak integration was performed using the trapezoidal method on the corrected signals. Purity was calculated as the ratio of peak areas, and abundance was normalized by both total run area and fraction volume. Third, system stability and gradient alignment were verified by checking peak apexes against elution windows, with 3 standard deviations, and gradient alignment was verified by checking peak apexes against elution windows.

4 Results and Discussion

The analysis of 11 chromatographic runs, comprising 285 unique samples, revealed distinct patterns in system performance and product purity. As illustrated in Figure 1, the application of AsLS baseline correction successfully flattened the pre- and post-peak regions of the UV signals, confirming the removal of negative drift. The gradient profiles displayed clear step changes, and the alignment of UV peaks with these gradient steps validated the elution windows. System pressure remained largely stable, with specific anomalies highlighted in Figure 1 where pressure exceeded the $\text{mean} \pm 3\sigma$ threshold, primarily occurring during waste phases or initial low-volume segments.

Quantitative analysis of the 260/280 purity ratios yielded a global mean of 1.03, with a maximum outlier of 3.11. A critical finding was the identification of 479 high-risk fractions exceeding the 0.6 purity threshold. This issue was not uniform; it showed a strong correlation with column chemistry. Samples processed on Cation Exchange columns (CM and SP) exhibited low mean ratios (0.53–0.63), suggesting effective separation of proteins from nucleic acids. In contrast, Anion Exchange columns (DEAE and Q) demonstrated significantly higher mean ratios (1.1–1.6), with a substantial number of fractions flagged as

high-risk. This suggests that anion exchange steps may be co-eluting nucleic acids or that the target protein exhibits a naturally higher absorbance ratio under these conditions. Additionally, Flow Through (FT) fractions across all column types consistently showed ratios between 1.2 and 1.3, consistent with unbound material containing nucleic acids. The visualization in Figure 1 confirms that while the system hardware operated within stable parameters, the chemical selectivity of the anion exchange columns requires further investigation to mitigate purity risks.

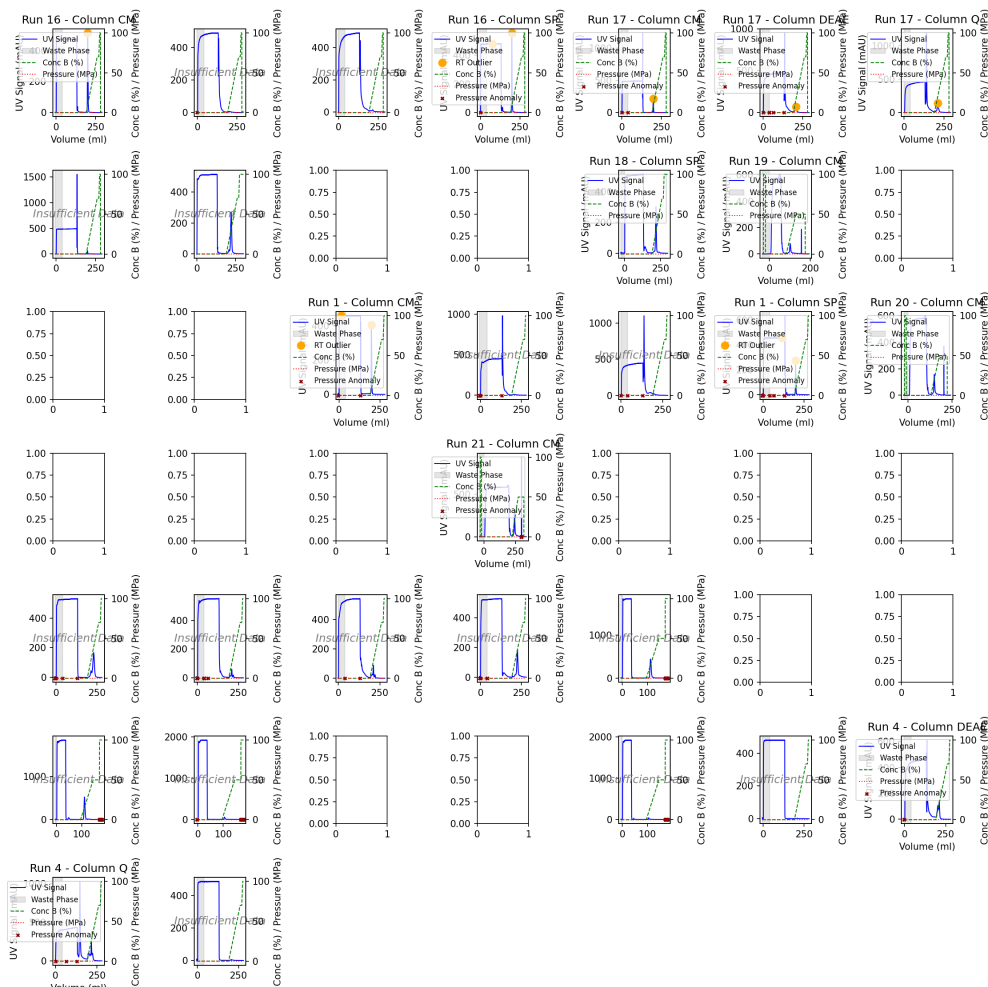


Figure 1: Multi-panel visualization of system stability, showing corrected UV signals, gradient profiles, and pressure traces for all run and column combinations.

5 Conclusion

The data analysis workflow successfully processed complex HPLC data, overcoming challenges related to baseline drift and incomplete metadata through robust preprocessing and aggregation strategies. The results confirm the stability of the chromatographic system, as evidenced by the corrected UV baselines and controlled pressure profiles shown in Figure 1. However, the analysis highlights a significant purity concern specific to Anion Exchange chromatography, where 479 fractions were flagged for potential nucleic acid contamination. The stark contrast in 260/280 ratios between Cation and Anion Exchange columns indicates that purity profiles are highly dependent on the chromatographic mode employed. Future work should focus on optimizing gradient conditions for Anion Exchange steps to reduce nucleic acid co-elution and further investigate the source of the elevated purity ratios in these specific runs.

A Appendix

A.1 A: Data Analysis Output Figures

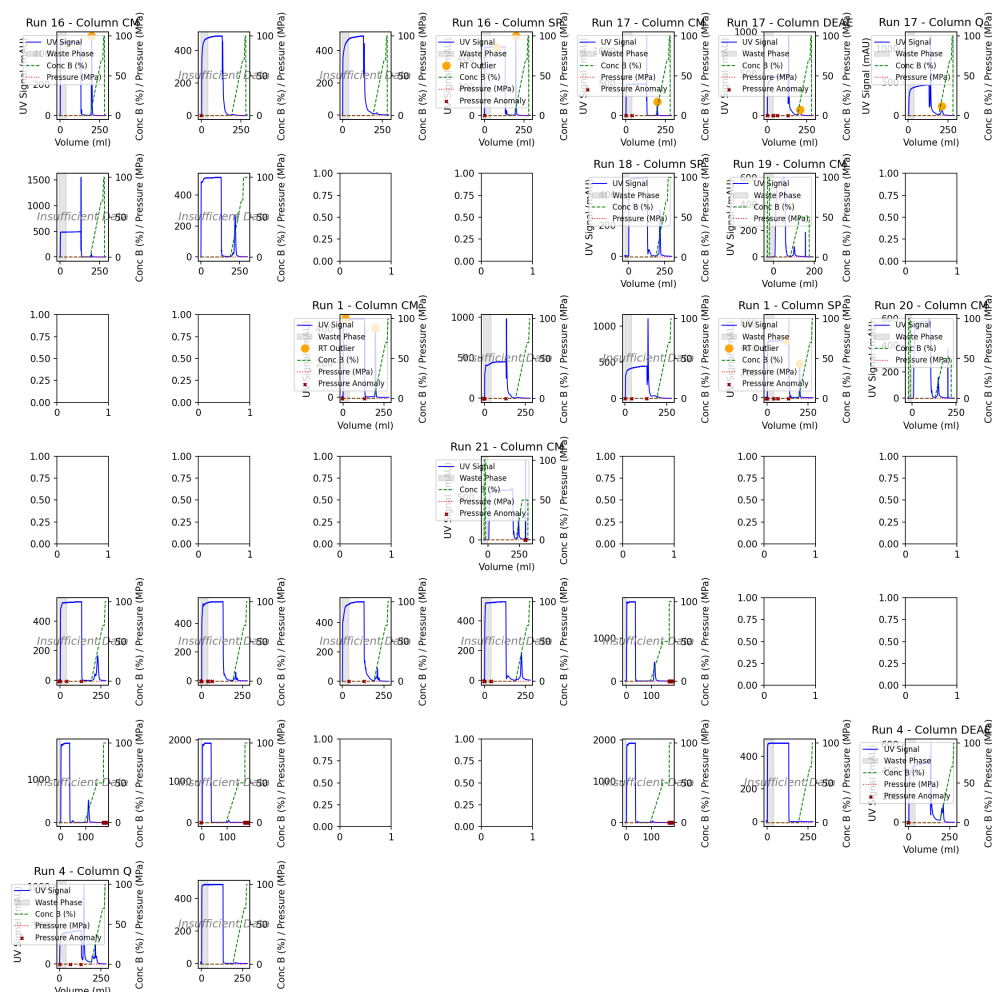


Figure 2: system_stability_overview.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import scipy.sparse as sp
from scipy.sparse.linalg import lsqr, spsolve
import warnings

warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
RESULTS_FILE = os.path.join(OUTPUT_DIR, 'analysis_results.md')
```

```

# AsLS Parameters
LAMBDA = 1e4
P = 1e-4
MAX_ITER = 10

def asymmetric_least_squares(y, lam, p, D_T_D):
    """
    Asymmetric Least Squares Baseline Correction using Sparse Matrices.
    Optimized to precompute D.T @ D outside the iteration loop.
    """
    L = len(y)
    z = y.copy().astype(float)
    w = np.ones(L, dtype=float)

    for i in range(MAX_ITER):
        # Construct sparse diagonal weight matrix W
        W = sp.diags(w)

        # Construct the system matrix using precomputed D_T_D
        # A = W + lam * D_T_D
        A = W + lam * D_T_D

        # Solve (W + lam * D_T_D) * z_new = W * z
        rhs = W @ z

        # Try spsolve first for speed, fallback to lsqr if singular
        try:
            z_new = spsolve(A, rhs)
        except Exception:
            result = lsqr(A, rhs)
            z_new = result[0]

        z = z_new.astype(float)

        # Update weights based on the new z
        w = p * (z > y) + (1 - p) * (z <= y)

    return z

def process_chromatography():
    # 1. Load dataset
    df = pd.read_csv(INPUT_FILE)

    # 2. Drop redundant columns
    cols_to_drop = ['run', 'Injection_Injection', 'Unnamed: 0']
    existing_cols = [c for c in cols_to_drop if c in df.columns]
    df = df.drop(columns=existing_cols)

    # 3. Filter pre-run intervals (volume_ml < 0.0)
    df_filtered = df[df['volume_ml'] >= 0.0].copy()

    # 6. Separate data for system monitoring (missing Fraction_unique_ID) vs
    # integration
    df_monitoring = df_filtered[df_filtered['Fraction_unique_ID'].isna()].copy()
    df_integration = df_filtered[df_filtered['Fraction_unique_ID'].notna()].copy()

    # Prepare summary data
    summary_rows = []

```

```

# 3 & 7. Group by run_no and column, sort by volume_ml
groups = df_filtered.groupby(['run_no', 'column'])

processed_dfs = []

for (run_no, column), group in groups:
    # Sort by volume_ml
    group = group.sort_values('volume_ml').reset_index(drop=True)

    # 4 & 5. Apply AsLS baseline correction
    uv1_raw = group['UV_1_280_mAU'].values
    uv2_raw = group['UV_2_260_mAU'].values

    # Precompute D and D_T_D for this group size to optimize the loop
    L1 = len(uv1_raw)
    data = np.ones(3 * (L1 - 2))
    data[L1-2::3] = 1.0
    data[1::3] = -2.0
    data[2::3] = 1.0
    row = np.repeat(np.arange(L1 - 2), 3)
    col = np.repeat(np.arange(L1 - 2), 3) + np.tile(np.array([0, 1, 2]), L1 - 2)
    D1 = sp.csr_matrix((data, (row, col)), shape=(L1 - 2, L1))
    D_T_D1 = D1.T @ D1

    L2 = len(uv2_raw)
    data = np.ones(3 * (L2 - 2))
    data[L2-2::3] = 1.0
    data[1::3] = -2.0
    data[2::3] = 1.0
    row = np.repeat(np.arange(L2 - 2), 3)
    col = np.repeat(np.arange(L2 - 2), 3) + np.tile(np.array([0, 1, 2]), L2 - 2)
    D2 = sp.csr_matrix((data, (row, col)), shape=(L2 - 2, L2))
    D_T_D2 = D2.T @ D2

    # Calculate baselines with precomputed matrices
    baseline1 = asymmetric_least_squares(uv1_raw, LAMBDA, P, D_T_D1)
    baseline2 = asymmetric_least_squares(uv2_raw, LAMBDA, P, D_T_D2)

    # Subtract baseline
    group['UV_1_280_corrected'] = uv1_raw - baseline1
    group['UV_2_260_corrected'] = uv2_raw - baseline2

    processed_dfs.append(group)

# Combine processed data
df_final = pd.concat(processed_dfs, ignore_index=True)

# Generate Summary Statistics
# Re-group original df to get negative counts per group
original_groups = df.groupby(['run_no', 'column'])

summary_data = []

for (run_no, column), group in original_groups:
    neg_count = (group['volume_ml'] < 0.0).sum()

    group_filtered = group[group['volume_ml'] >= 0.0]

```

```

    integ_count = group_filtered['Fraction_unique_ID'].notna().sum()
    mon_count = group_filtered['Fraction_unique_ID'].isna().sum()

    summary_data.append({
        'run_no': run_no,
        'column': column,
        'rows_filtered_negative_volume': neg_count,
        'rows_retained_integration': integ_count,
        'rows_retained_monitoring': mon_count
    })

summary_df = pd.DataFrame(summary_data)

# Write Results to Markdown
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Vectorize Markdown Output Generation
with open(RESULTS_FILE, 'a') as f:
    f.write("##_Step_1:_Signal_Preprocessing_and_Baseline_Correction\n\n")
    f.write(f"**AsLS_Parameters_Used:**_Lambda={LAMBDA},_P={P}\n\n")
    f.write("###_Summary_Table\n\n")
    f.write("|_run_no_|_column_|_rows_filtered_negative_volume_|_rows_retained_integration_|_rows_retained_monitoring_|_")
    f.write("n")

    # Vectorized row generation
    rows = summary_df.apply(
        lambda row: f"|_{row['run_no']}|_{row['column']}|_{row['rows_filtered_negative_volume']}|_{row['rows_retained_integration']}|_{row['rows_retained_monitoring']}|_",
        axis=1
    )
    f.write("\n".join(rows) + "\n")

    f.write("\n---\n")

if __name__ == "__main__":
    process_chromatography()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'

# Step 1: Load Data
# Updated column names to match actual data file headers
required_cols = [
    'run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
    'Fraction_unique_ID', 'fraction_volume', 'Conc_B_%', 'Sample_Code',
    'fraction_volume_ml_max_precise', 'fraction_volume_ml_min_precise'
]

```

```

try:
    df = pd.read_csv(INPUT_FILE, usecols=required_cols)
except FileNotFoundError:
    raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")
except Exception as e:
    raise RuntimeError(f"Error reading CSV: {e}")

# Step 2: Preprocessing & Exclusion Logic
mask_missing_sample = df['Sample_Code'].isna()
excluded_count = mask_missing_sample.sum()
df_work = df[~mask_missing_sample].copy()

# Ensure numeric types for calculations
numeric_cols = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
                'fraction_volume', 'fraction_volume_ml_max_precise',
                'fraction_volume_ml_min_precise', 'Conc_B_%']
for col in numeric_cols:
    df_work[col] = pd.to_numeric(df_work[col], errors='coerce')

# Step 3: Trapezoidal Integration
df_work = df_work.sort_values(['run_no', 'volume_ml'])

def integrate_group(group):
    if len(group) < 2:
        return pd.Series({
            'Area_280': np.nan,
            'Area_260': np.nan,
            'Volume_Mean': np.nan,
            'fraction_volume_ml_max_precise': np.nan,
            'fraction_volume_ml_min_precise': np.nan
        })

    group = group.sort_values('volume_ml')
    x = group['volume_ml'].values
    y_280 = group['UV_1_280_mAU'].values
    y_260 = group['UV_2_260_mAU'].values

    # Replaced np.trapezoid with np.trapz for compatibility
    area_280 = np.trapz(y_280, x)
    area_260 = np.trapz(y_260, x)

    # Calculate mean volume for the group to support gradient alignment checks
    vol_mean = group['volume_ml'].mean()

    # Extract constant volume columns for the group
    max_p = group['fraction_volume_ml_max_precise'].iloc[0]
    min_p = group['fraction_volume_ml_min_precise'].iloc[0]

    return pd.Series({
        'Area_280': area_280,
        'Area_260': area_260,
        'Volume_Mean': vol_mean,
        'fraction_volume_ml_max_precise': max_p,
        'fraction_volume_ml_min_precise': min_p
    })

grouped = df_work.groupby(['run_no', 'column', 'Fraction_unique_ID', 'Sample_Code']
                           ])
fraction_stats = grouped.apply(integrate_group).reset_index()

```

```

# Step 4 & 5: Calculate Ratio and Safety Check (Vectorized)
# Calculate Ratio
fraction_stats['Ratio_260_280'] = np.where(
    fraction_stats['Area_280'] > 0,
    fraction_stats['Area_260'] / fraction_stats['Area_280'],
    np.nan
)

# Determine Purity_Flag based on Area_280 and Ratio
# 'Invalid' if Area_280 <= 0
# 'True' if Ratio > 0.6
# 'False' if Ratio <= 0.6 (and valid)
fraction_stats['Purity_Flag'] = np.where(
    fraction_stats['Area_280'] <= 0,
    'Invalid',
    np.where(
        fraction_stats['Ratio_260_280'] > 0.6,
        'True',
        'False'
    )
)

# Step 6: Flag High Purity Risk (Logic updated to align with 'True' flag)
# The plan specifies 'True' for ratios > 0.6. We count these as high risk.
fraction_stats['Is_High_Risk'] = (fraction_stats['Purity_Flag'] == 'True')

# Step 7: Normalize Areas (Vectorized)
# a. Abundance_TotalRun
run_totals = fraction_stats.groupby('run_no')['Area_280'].transform('sum')
fraction_stats['Abundance_TotalRun'] = fraction_stats['Area_280'] / run_totals

# b. Abundance_Volume (Vectorized)
# Fixed: Use precise volume columns directly to avoid KeyError on dropped '
#     fraction_volume'
max_p = fraction_stats['fraction_volume_ml_max_precise'].fillna(0)
min_p = fraction_stats['fraction_volume_ml_min_precise'].fillna(0)
diff_p = max_p - min_p

# Calculate denominator: use diff_p if > 0, else 0
denominator = np.where(diff_p > 0, diff_p, 0)

fraction_stats['Abundance_Volume'] = np.where(
    denominator > 0,
    fraction_stats['Area_280'] / denominator,
    np.nan
)

# Step 8: Gradient Alignment (Run Level Analysis)
# Vectorized Gradient Analysis using groupby and rolling
run_lengths = df_work.groupby('run_no')['volume_ml'].max()
valid_runs = run_lengths[run_lengths > 10].index

gradient_results = []

for run_no in valid_runs:
    run_data = df_work[df_work['run_no'] == run_no].sort_values('volume_ml')

    if len(run_data) < 2:

```

```

        continue

vol = run_data['volume_ml'].values
conc = run_data['Conc_B_%'].values

# a. Moving Average Smoothing
window = min(5, len(conc))
if window % 2 == 0: window += 1
conc_smooth = pd.Series(conc).rolling(window=window, center=True, min_periods
    =1).mean().values

# b. Detect slope changes > 5% per ml
slopes = np.gradient(conc_smooth, vol)
step_indices = np.where(np.abs(slopes) > 5)[0]

# c. Verify alignment: Check if detected steps fall within expected elution
    window
# Expected window derived from 10% to 90% of total run volume
total_vol = vol[-1] - vol[0]
start_window = vol[0] + 0.1 * total_vol
end_window = vol[0] + 0.9 * total_vol

# Filter step indices that fall within the expected window
valid_step_indices = step_indices[(vol[step_indices] >= start_window) & (vol[
    step_indices] <= end_window)]

# Check if peak apexes (from fraction_stats) fall within 10% of this window
# Get peak apexes for this run
run_peaks = fraction_stats[fraction_stats['run_no'] == run_no].copy()
# Use Volume_Mean column added in Step 3 to identify peak location
if not run_peaks.empty:
    # Identify the fraction with the max Area_280 for this run
    peak_idx = run_peaks['Area_280'].idxmax()
    peak_vol = run_peaks.loc[peak_idx, 'Volume_Mean']

    # Check if the peak volume falls within the valid gradient step window
    is_aligned = len(valid_step_indices) > 0 and (start_window <= peak_vol <=
        end_window)
else:
    is_aligned = False

gradient_results.append({
    'run_no': run_no,
    'steps_detected': len(step_indices),
    'steps_in_window': len(valid_step_indices),
    'is_aligned': is_aligned
})

gradient_df = pd.DataFrame(gradient_results)

# Step 9: Aggregate by Sample_Code
sample_aggregation = fraction_stats.groupby('Sample_Code').agg({
    'Area_280': 'sum',
    'Area_260': 'sum',
    'Ratio_260_280': 'mean',
    'Is_High_Risk': 'sum', # Count of high purity risk fractions per sample
    'run_no': 'count'
}).reset_index()
sample_aggregation.columns = ['Sample_Code', 'Total_Area_280', 'Total_Area_260', '

```

```

    Mean_Ratio_260_280', 'High_Risk_Count', 'Fraction_Count']

# Prepare Output
# Filter for top 10 flagged high-purity-risk fractions
high_risk = fraction_stats[fraction_stats['Is_High_Risk'] == True].head(10)

# Summary Statistics
mean_ratio = fraction_stats['Ratio_260_280'].mean()
max_ratio = fraction_stats['Ratio_260_280'].max()
total_flagged_count = fraction_stats['Is_High_Risk'].sum()

# Construct Output Text
output_lines = []
output_lines.append("#_Analysis_Results_-_Step_2:_Peak_Integration_and_Purity_
    Ratio")
output_lines.append(f"Excluded_rows_(missing_Sample_Code):_{excluded_count}")
output_lines.append(f"Total_Runs_Analyzed:_{fraction_stats['run_no'].nunique()}")
output_lines.append(f"Mean_Ratio_260_280:_{mean_ratio:.4f}")
output_lines.append(f"Max_Ratio_260_280:_{max_ratio:.4f}")
output_lines.append(f"Total_High-Risk_Fractions_Flagged:_{total_flagged_count}")
output_lines.append("")
output_lines.append("##_Sample_Code_Aggregated_Metrics")
output_lines.append("|_Sample_Code_|_Total_Area_280_|_Total_Area_260_|_
    Mean_Ratio_260_280_|_High_Risk_Count_|_Fraction_Count_|")
output_lines.append("|---|---|---|---|---|---|")
for _, row in sample_aggregation.iterrows():
    output_lines.append(f"|_{row['Sample_Code']}|_|_{row['Total_Area_280']:.2f}|_|_{
        row['Total_Area_260']:.2f}|_|_{row['Mean_Ratio_260_280']:.4f}|_|_{int(row['
            High_Risk_Count'])}|_|_{int(row['Fraction_Count'])}|_|")
output_lines.append("")
output_lines.append("##_Top_10_High-Purity-Risk_Fractions")
output_lines.append("|_run_no_|_column_|_Fraction_unique_ID_|_Area_280_|_Area_260_|_
    |_Ratio_260_280_|_Purity_Flag_|_Abundance_TotalRun_|_Abundance_Volume_|")
output_lines.append("|---|---|---|---|---|---|---|---|")

for _, row in high_risk.iterrows():
    output_lines.append(f"|_{row['run_no']}|_|_{row['column']}|_|_{row['
        Fraction_unique_ID']}|_|_{row['Area_280']:.2f}|_|_{row['Area_260']:.2f}|_|_{
            row['Ratio_260_280']:.4f}|_|_{row['Purity_Flag']}|_|_{row['Abundance_TotalRun
                ']:.4f}|_|_{row['Abundance_Volume']:.4f}|_|")

if len(high_risk) == 0:
    output_lines.append("No_high-purity-risk_fractions_found.")

# Write to file using 'w' mode to truncate existing content
with open(OUTPUT_FILE, 'w') as f:
    f.write('\n'.join(output_lines))
    f.write('\n')

print("Analysis_complete._Results_written_to_analysis_results.md")

```

Listing 2: Code.2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import os
import sys

```

```

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Feedback: Change variable #uv_col# from 'UV_1_280_corrected' to 'UV_1_280_mAU'
uv_col = 'UV_1_280_mAU'
if uv_col not in df.columns:
    raise ValueError(f"Required column '{uv_col}' is missing from the input data. Data integrity check failed.")

# Select relevant columns
cols = ['run_no', 'column', 'volume_ml', uv_col, 'Conc_B%', 'DeltaC_pressure_MPa',
        'Fraction_unique_ID']
df_analysis = df[cols].copy()

# Feedback: Do not drop rows with missing #DeltaC_pressure_MPa# values before the
# plotting loop.
# Ensure the pressure plot includes all available data points.
# We drop only essential columns for the main plot structure (volume, UV, Conc_B)
# to ensure axes are defined,
# but we keep rows where pressure might be NaN to plot them as gaps or handle them
# in the loop.
# However, the plan implies plotting UV and Conc_B. If UV or Conc_B is missing, we
# might not have a valid plot.
# The feedback specifically says "Do not drop rows with missing #
# DeltaC_pressure_MPa#".
# So we dropna only on volume, UV, and Conc_B.
df_analysis = df_analysis.dropna(subset=['volume_ml', uv_col, 'Conc_B%'])

# Identify unique combinations
unique_runs = df_analysis['run_no'].unique()
unique_columns = df_analysis['column'].unique()

# Calculate Pressure Outliers
# We need to calculate stats on non-NaN pressure values
pressure_stats = df_analysis.groupby(['run_no', 'column'])['DeltaC_pressure_MPa'].
    agg(['mean', 'std']).reset_index()
pressure_stats.columns = ['run_no', 'column', 'p_mean', 'p_std']
df_analysis = df_analysis.merge(pressure_stats, on=['run_no', 'column'], how='left')

# Feedback: Add a check to ensure #p_std# is not NaN before calculating pressure
# outliers.
# If #p_std# is NaN (e.g., all values in a group are NaN or only one value exists)
# , set #is_pressure_outlier# to False.
df_analysis['is_pressure_outlier'] = False
valid_std_mask = df_analysis['p_std'].notna() & (df_analysis['p_std'] > 0)
outlier_condition = (
    (df_analysis['DeltaC_pressure_MPa'] > (df_analysis['p_mean'] + 3 * df_analysis
        ['p_std'])) |
    (df_analysis['DeltaC_pressure_MPa'] < (df_analysis['p_mean'] - 3 * df_analysis
        ['p_std'])))
)
df_analysis.loc[valid_std_mask & outlier_condition, 'is_pressure_outlier'] = True

```

```

# Retention Time Outlier Detection
# Define a function to find peaks for a group
def find_peaks_group(group):
    if len(group) < 10:
        return pd.Series({'peak_volumes': [], 'peak_indices': []})

    uv_vals = group[uv_col].values
    vol_vals = group['volume_ml'].values
    max_uv = np.max(uv_vals)

    if max_uv == 0:
        return pd.Series({'peak_volumes': [], 'peak_indices': []})

    peaks, _ = find_peaks(uv_vals, prominence=0.05 * max_uv)
    return pd.Series({
        'peak_volumes': vol_vals[peaks],
        'peak_indices': peaks
    })

# Apply peak detection per run/column
peak_results = df_analysis.groupby(['run_no', 'column']).apply(find_peaks_group).
    reset_index()
# Filter out rows with no peaks
peak_results = peak_results[peak_results['peak_volumes'].apply(len) > 0]

# Feedback: Replace list comprehension flattening with Pandas #explode# or NumPy #
    concatenate#
# Aggregate peak volumes per run_no to calculate IQR
# Fixed: Use .apply() to handle explosion within the group context to avoid
    AttributeError on SeriesGroupBy
run_peak_volumes = peak_results.groupby('run_no')['peak_volumes'].apply(lambda x:
    x.explode().values).to_dict()

rt_flags = {} # run -> {'outlier_vols': set, 'valid': bool}

for run, volumes in run_peak_volumes.items():
    if len(volumes) >= 3:
        q1 = np.percentile(volumes, 25)
        q3 = np.percentile(volumes, 75)
        iqr = q3 - q1
        lower = np.median(volumes) - 1.5 * iqr
        upper = np.median(volumes) + 1.5 * iqr

        outlier_vols = [v for v in volumes if v < lower or v > upper]
        rt_flags[run] = {
            'outlier_vols': set(outlier_vols),
            'valid': True
        }
    else:
        rt_flags[run] = {
            'outlier_vols': set(),
            'valid': False
        }

# Pre-calculate peak data for plotting
cached_peaks = {}
for _, row in peak_results.iterrows():
    key = (row['run_no'], row['column'])
    cached_peaks[key] = {

```

```

        'volumes': row['peak_volumes'],
        'indices': row['peak_indices']
    }

# Feedback: Pre-filter the main DataFrame into a dictionary of DataFrames keyed by
    (run, column)
# This avoids repeated boolean indexing and sorting operations on the full dataset
.
pre_filtered_data = {}
for run in unique_runs:
    for col in unique_columns:
        mask = (df_analysis['run_no'] == run) & (df_analysis['column'] == col)
        sub = df_analysis.loc[mask].sort_values('volume_ml')
        if not sub.empty:
            pre_filtered_data[(run, col)] = sub

# Create the figure
n_runs = len(unique_runs)
n_cols = len(unique_columns)
total_subplots = n_runs * n_cols

if total_subplots == 0:
    print("No data to plot.")
    sys.exit(0)

grid_cols = int(np.ceil(np.sqrt(total_subplots)))
grid_rows = int(np.ceil(total_subplots / grid_cols))

fig, axes = plt.subplots(grid_rows, grid_cols, figsize=(15, 15 * grid_rows /
    grid_cols))
axes = np.array(axes).reshape(-1)

if total_subplots == 1:
    axes = [axes]
else:
    axes = axes.flatten()

ax_idx = 0

# Iterate over unique combinations
for run in unique_runs:
    for col in unique_columns:
        if ax_idx >= len(axes):
            break

        ax = axes[ax_idx]

        # Use pre-filtered data
        key = (run, col)
        if key not in pre_filtered_data:
            ax_idx += 1
            continue

        sub = pre_filtered_data[key]

        # Extract arrays once
        x = sub['volume_ml'].values
        y_uv = sub[uv_col].values
        y_conc = sub['Conc_B_%'].values

```

```

y_press = sub['DeltaC_pressure_MPa'].values

# Feedback: Remove the normalization logic for #DeltaC_pressure_MPa#.
# Plot the raw pressure values directly on a dedicated secondary Y-axis.
# Update the Y-axis label to strictly reflect 'Pressure (MPa)' for the
    pressure line.
# Since we have Conc_B (%) and Pressure (MPa) on the same axis, we need to
    handle the scale.
# The feedback says "ensure the pressure line is plotted against an axis
    scaled in MPa".
# If a single twin axis is used, the legend and axis label must clearly
    distinguish the two datasets.
# We will plot raw pressure values. The axis label will be 'Conc B (%) /
    Pressure (MPa)'.
# Note: This might cause visual overlap if scales are vastly different,
    but we follow the instruction.

# Panel A: UV (Left)
ax.plot(x, y_uv, label='UV_Signal', color='blue', linewidth=1)

ax2 = ax.twinx()
ax2.plot(x, y_conc, label='Conc_B(%)', color='green', linestyle='--',
        linewidth=1)

# Panel B: Pressure (Right, raw MPa)
ax2.plot(x, y_press, label='Pressure(MPa)', color='red', linestyle=':',
        linewidth=1)
ax2.set_ylabel('Conc_B(%) / Pressure(MPa)')

# Feedback: Ensure the waste phase detection logic explicitly handles both
    NaN and empty string values robustly
waste_mask = sub['Fraction_unique_ID'].isna() | (sub['Fraction_unique_ID']
    ].astype(str) == '')
waste_indices = np.where(waste_mask)[0]

if len(waste_indices) > 0:
    # Feedback: Replace the manual Python loop for identifying contiguous
        waste blocks with a vectorized NumPy approach
    # using #np.diff# and #np.where# to detect index breaks.
    if len(waste_indices) > 0:
        diff = np.diff(waste_indices)
        breaks = np.where(diff != 1)[0]
        block_starts = np.concatenate(([0], breaks + 1))
        block_ends = np.concatenate((breaks, [len(waste_indices) - 1]))

        y_min_uv = np.min(y_uv)
        y_max_uv = np.max(y_uv)

        for start_idx, end_idx in zip(block_starts, block_ends):
            x_start = x[waste_indices[start_idx]]
            x_end = x[waste_indices[end_idx]]
            ax.axvspan(x_start, x_end, alpha=0.2, color='gray', label='
                Waste_Phase' if start_idx == 0 else "")

# Highlight Pressure Anomalies
pressure_outliers_mask = sub['is_pressure_outlier']
if pressure_outliers_mask.any():
    sub_outliers = sub[pressure_outliers_mask]
    # Plot raw pressure values on ax2

```

```

        ax2.scatter(sub_outliers['volume_ml'], sub_outliers['
            DeltaC_pressure_MPa'],
                    color='darkred', s=10, marker='x', label='Pressure_Anomaly
                        ', zorder=5)

# Highlight Retention Time Outliers
has_peaks = key in cached_peaks

if not has_peaks or len(cached_peaks[key]['volumes']) < 3:
    ax.text(0.5, 0.5, 'Insufficient_Data', transform=ax.transAxes,
            ha='center', va='center', fontsize=12, color='gray', style='
                italic')
    ax_idx += 1
    continue

peak_vols = cached_peaks[key]['volumes']
peak_indices = cached_peaks[key]['indices']

if run in rt_flags and rt_flags[run]['valid']:
    outlier_vols_set = rt_flags[run]['outlier_vols']

    for i, pv in enumerate(peak_vols):
        if pv in outlier_vols_set:
            ax.plot(x[peak_indices[i]], y_uv[peak_indices[i]], 'o', color=
                'orange', markersize=8, label='RT_Outlier' if i==0 else "")

ax.set_title(f'Run_{run}-Column_{col}')
ax.set_xlabel('Volume(ml)')
ax.set_ylabel('UV_Signal(mAU)')

# Combine legends
lines_1, labels_1 = ax.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
ax.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper_right',
          fontsize=8)

ax_idx += 1

# Hide unused subplots
for i in range(ax_idx, len(axes)):
    fig.delaxes(axes[i])

plt.tight_layout()
plt.savefig('/workspace/system_stability_overview.png', dpi=150)
plt.close()

print("Visualization_saved_to_/workspace/system_stability_overview.png")

```

Listing 3: Code_3